OS

Definition

An operating system (OS) is the software layer that acts as an interface between user applications and the computer hardware. Its primary functions include:

- **Resource Management:** Allocates CPU time, memory, and storage to various processes.
- **Process Management:** Creates, schedules, and terminates processes to ensure efficient multitasking.
- **Memory Management:** Controls access to RAM, ensuring that each process operates within its own isolated address space.
- **File System Management:** Organizes and manages data stored on disks, providing a structure for file access.
- **I/O Device Control:** Provides a unified interface for applications to interact with hardware devices like keyboards, monitors, and network adapters.
- **Security & Protection:** Regulates user access through authentication and guards against unauthorized system interaction.

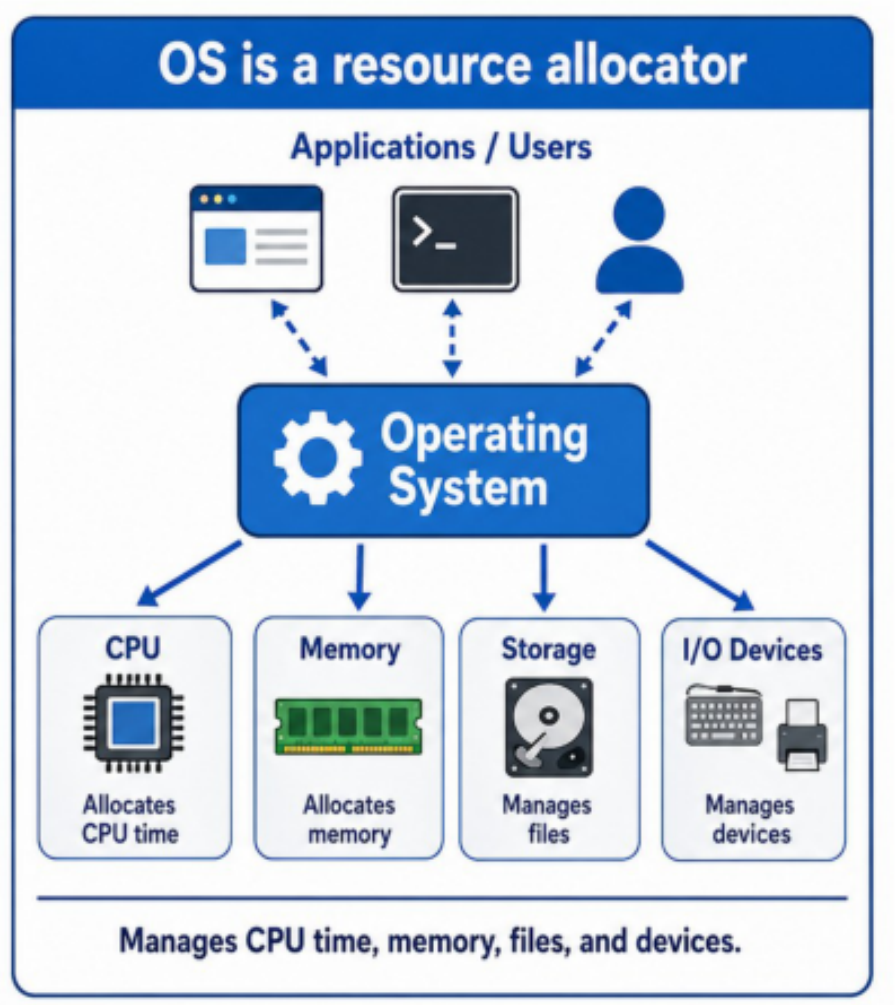

Figure 1: OS is a resource allocator

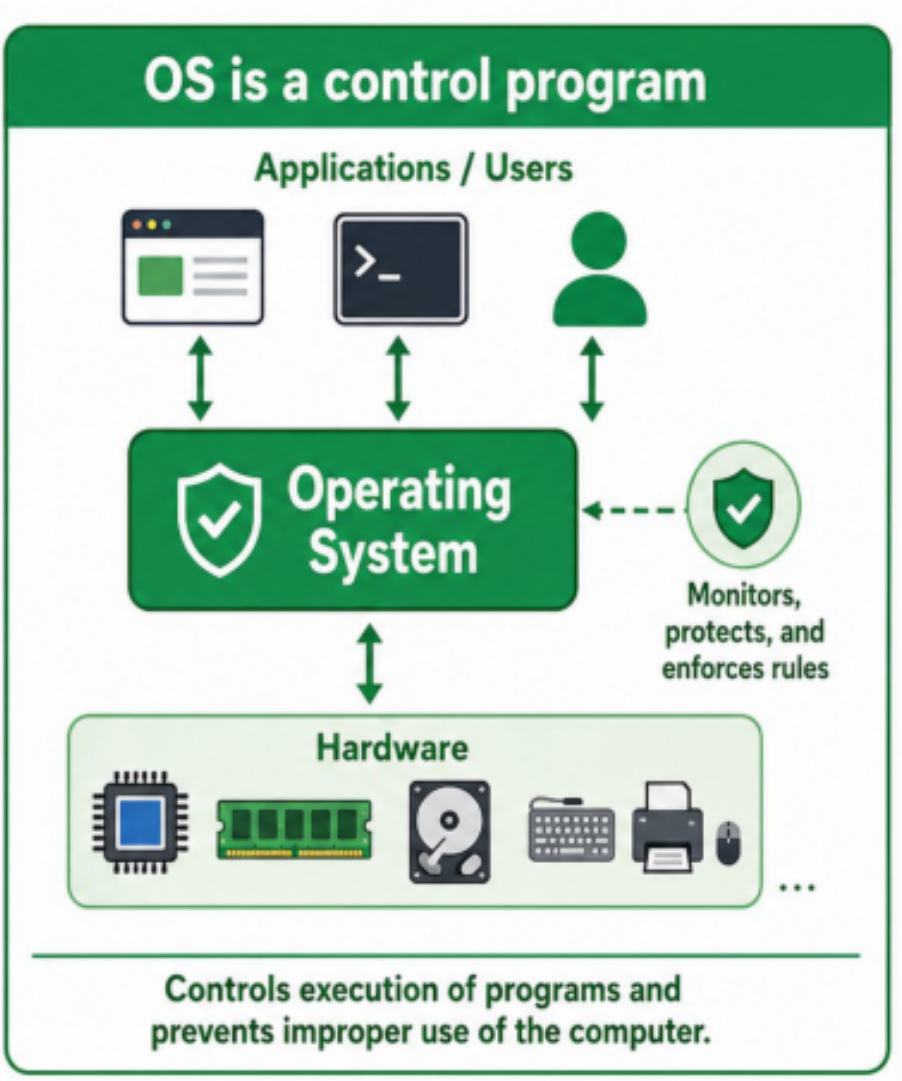

Figure 2: OS is a control program

OSTEP defines it in three steps which are:

- Virtualization
- Concurrency
- Parallelism

Computer System Structure

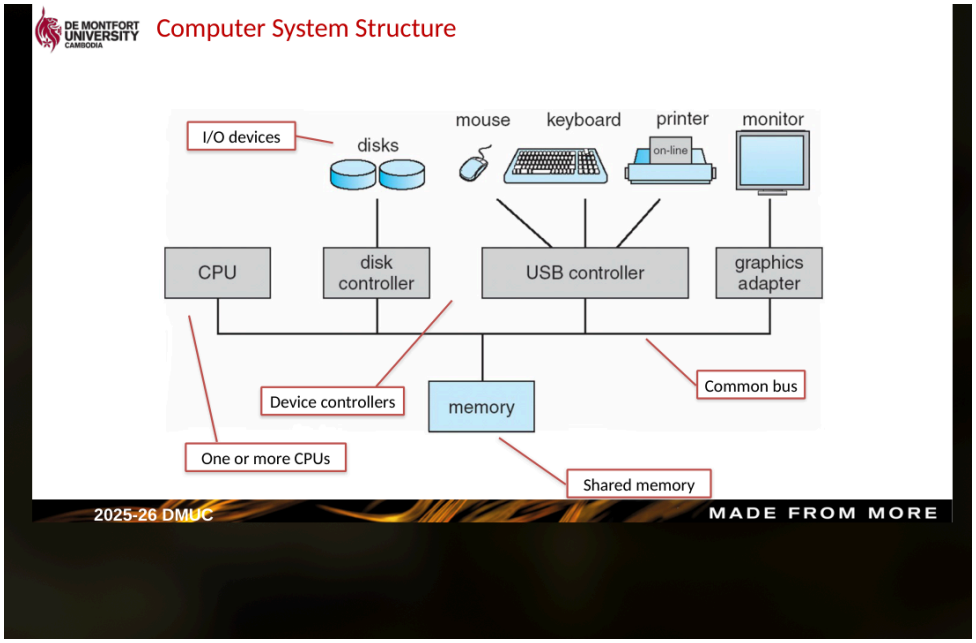

Figure 3: Computer System Structure

Interrupts

- OS is interrupt driven - an interrupt is a signal sent to the OS:
  - Hardware interrupt by one of the devices
  - Software interrupt generated by user request
  - Trap or exception generated by an error

Dual-Mode Operation

- Dual-mode operation allows the OS to protect itself, as well as other components.
  - User mode and kernel mode

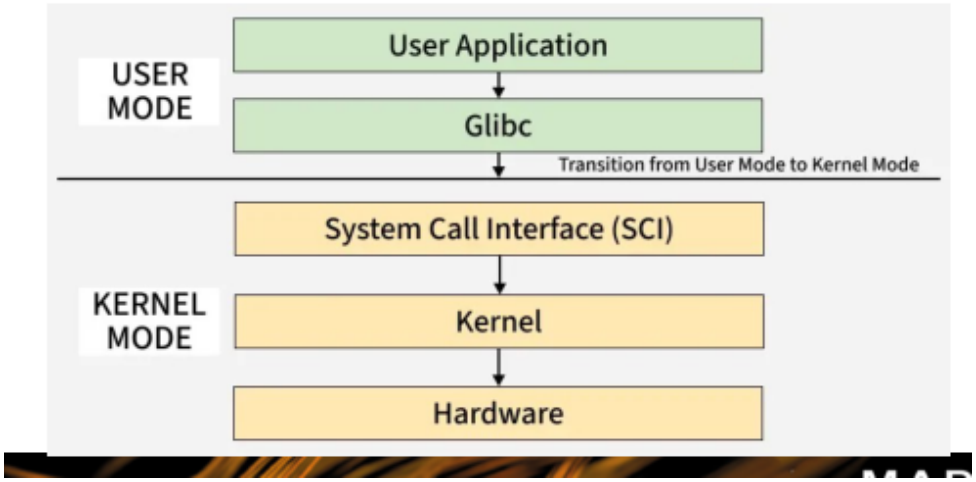

Figure 4: User mode and Kernel mode

Kernel Mode

- In kernel mode, the operating system has unrestricted access to the underlying hardware and memory.
  - Can execute privileged instructions that are not available to user programs.

- Can manage system resources such as CPU scheduling, memory allocation, and I/O devices.
- Can handle interrupts, exceptions, and traps directly.
- Executes with full authority to modify the system state, including kernel data structures.

User Mode

- In user mode, the operating system restricts the program's access to hardware and system resources to prevent system crashes or malicious activity.
  - Programs have limited access to memory and cannot directly access I/O devices.
  - Privileged instructions are prohibited; any attempt to execute them triggers a trap or exception that the kernel must handle.
  - Programs must request system services via system calls to perform restricted operations.
  - The mode provides a protected environment where applications can run safely without interfering with the kernel or other running processes.

Comparison

Feature	**User Mode**	**Kernel Mode**
Access	Restricted	Unrestricted
Instructions	Non-privileged only	Privileged and non-privileged
Memory	Isolated address space	Full system memory
Services	Request via system calls	Direct access to hardware
Stability	Safe/Protected	Critical/Requires care

Kernel Architectures

- **Monolithic Kernel**: All OS services (file system, device drivers, memory management, etc.) run in the same kernel address space. This provides high performance due to minimal communication overhead, but a bug in one component can crash the entire system.
  - Examples: Linux, FreeBSD, MS-DOS.
- **Microkernel**: Only the most essential functions (minimal mechanisms for IPC, virtual memory, and scheduling) run in the kernel mode. Other services run in user space as separate processes. This design is highly modular and stable, though communication between services can induce latency.
  - Examples: QNX, L4, MINIX.
- **Hybrid Kernel**: A compromise between monolithic and microkernel designs. It maintains the core performance of a monolithic system while incorporating some modularity from a microkernel, often by running certain services (like graphics drivers) in user space while keeping others in the kernel.
  - Examples: Windows NT (and its successors like Windows 10/11), macOS (XNU kernel).